MODUL PRAKTIKUM

PEMROGRAMAN BERORIENTASI OBJEK

PENYUSUN:

Timotheus Ferdinand Tjondrojo (NIM: 105222019)

Yan Stephen Christian Immanuel Hutagalung (NIM: 105222010)

LABORATORIUM KOMPUTER

PROGRAM STUDI ILMU KOMPUTER

FAKULTAS SAINS DAN ILMU KOMPUTER

UNIVERSITAS PERTAMINA

2026

Encapsulation

PRE-TEST (Durasi Maksimal: 20 Menit)

Buatlah sebuah prototipe sederhana sistem inventaris menggunakan konsep OOP. Buatlah sebuah package bernama minimarket. Di dalam package tersebut, rancanglah sebuah kelas parent bernama Produk yang memiliki dua atribut dasar: namaProduk (String) dan harga (double). Buatlah sebuah konstruktor untuk mengisi kedua atribut tersebut, serta sediakan fungsi tampilkanDetail() untuk mencetak nama dan harga produk.

Selanjutnya, buatlah sebuah subclass bernama MakananRingan yang mewarisi kelas Produk. Kelas ini memiliki satu atribut spesifik tambahan yaitu rasa (String). Buatlah konstruktor untuk kelas ini dengan memanfaatkan keyword super() untuk mengisi nama dan harga. Kemudian, lakukan override pada fungsi tampilkanDetail() agar mencetak atribut rasa, dengan tetap memanggil fungsi penampil milik superclass menggunakan super.tampilkanDetail(). Lalu, buatlah kelas Main. Di dalamnya, cukup instansiasikan satu buah objek MakananRingan dan panggil fungsi tampilkanDetail() miliknya.

1. Package

1.1 Pengertian dan Fungsi Package

Dalam pemrograman Java, Package adalah sebuah direktori atau folder yang digunakan untuk mengelompokkan kelas (*class*), antarmuka (*interface*), dan *sub-package* yang memiliki fungsi atau kategori sejenis.

Fungsi utama Package:

1. **Mencegah Konflik Nama (*Name Collision*):** Kita bisa memiliki dua kelas dengan nama yang sama, asalkan mereka berada di package yang berbeda (misal: frontend.Button dan backend.Button).
2. **Pengendalian Akses (*Access Control*):** Package bekerja sama dengan *access modifier* (seperti protected dan *default/package-private*) untuk membatasi bagian kode mana yang bisa diakses oleh kelas di luar package.
3. **Keteraturan:** Memudahkan pencarian dan pengelolaan file seiring berkembangnya ukuran proyek aplikasi.

1.2 Hirarki dan Deklarasi Package

Nama package seringkali merepresentasikan struktur folder berjenjang, dipisahkan oleh tanda titik (.). Konvensi umum penamaan package menggunakan huruf kecil semua dan sering dibalik dari nama domain perusahaan (misalnya: com.perusahaan.namaaplikasi).

Aturan Deklarasi:

- Statement package **wajib** menjadi baris pertama (yang bukan komentar) dalam file .java.
- Satu file .java hanya boleh memiliki maksimal satu statement package.

Contoh Deklarasi:

Jika kode berikut ada di file Karyawan.java, maka file tersebut **harus** disimpan secara fisik dalam struktur folder: src/perusahaan/hrd/Karyawan.java.

```
// Baris pertama mendeklarasikan posisi package
package perusahaan.hrd;

public class Karyawan {
   public String nama;

   public void bekerja() {
      System.out.println("Karyawan sedang bekerja.");
   }
}
```

1.3 Penggunaan import antar Package

Jika sebuah kelas ingin menggunakan kelas lain yang berada di *package yang berbeda*, kita harus menggunakan kata kunci import.

(Pengecualian: kelas-kelas dalam package java.lang seperti String atau System diimpor secara otomatis).

Sintaks:

```
// Mengimpor satu kelas spesifik secara langsung (Disarankan)
import nama_package.NamaKelas;

// Mengimpor SELURUH kelas di dalam package tersebut (Menggunakan wildcard *)
import nama_package.*;
```

2. Inheritance

2.1 Konsep Dasar Pewarisan (IS-A Relationship)

Inheritance atau pewarisan adalah pilar utama dalam *Object-Oriented Programming* (OOP) di mana sebuah kelas baru dapat diciptakan dengan mengadopsi (mewarisi) atribut dan *method* dari kelas yang sudah ada.

Pewarisan menciptakan hubungan **"IS-A" (Adalah sebuah)**. Contoh:

- Kucing **adalah sebuah** Hewan (*Cat IS-A Animal*).
- Mobil **adalah sebuah** Kendaraan (*Car IS-A Vehicle*).

Istilah dalam Inheritance:

- **Superclass / Parent Class / Base Class:** Kelas utama yang propertinya diwariskan.
- **Subclass / Child Class / Derived Class:** Kelas baru yang mewarisi sifat dari kelas utama. Sebuah subclass bisa menambahkan atribut/method baru yang spesifik.

2.2 Aturan Pewarisan dengan extends

Di Java, kelas anak menggunakan kata kunci extends untuk mewarisi kelas induk.

- **Single Inheritance:** Java hanya mengizinkan *single inheritance* untuk kelas (satu kelas anak hanya boleh punya **satu** *superclass* langsung).
- **Visibilitas:** Atribut dan method yang bersifat private **TIDAK** diwariskan ke subclass secara langsung. Jika ingin atribut tidak bisa diakses publik tapi tetap bisa dipakai oleh subclass, gunakan *modifier* protected.

2.3 Contoh Dasar Inheritance

```
// 1. Superclass
    class Hewan {
        String nama;

        public void makan() {
            System.out.println(nama + " sedang makan.");
        }
    }

    // 2. Subclass
    class Kucing extends Hewan {
        String warnaBulu;

        public void mengeong() {
            System.out.println(nama + " mengeong: Meow!");
        }
    }

    public class MainHewan {
        public static void main(String[] args) {
            Kucing garfield = new Kucing();

            garfield.nama = "Garfield";
            garfield.warnaBulu = "Oren";

            garfield.makan();   // Output: Garfield sedang makan.
            garfield.mengeong(); // Output: Garfield mengeong: Meow!
        }
    }
```

3. Super Keyword dan Method Overriding

Kata kunci super adalah referensi variabel yang merepresentasikan objek *superclass* secara langsung.

3.1 super untuk Mengakses Atribut yang Tertutup (*Variable Shadowing*)

Jika subclass memiliki variabel dengan nama yang **sama persis** dengan variabel di superclass, gunakan super.namaVariabel untuk mengakses variabel milik superclass.

```
class Kendaraan {
    int kecepatanMaksimal = 120;
}

class MobilBalap extends Kendaraan {
    int kecepatanMaksimal = 350;

    public void tampilkanKecepatan() {
        System.out.println("Kecepatan Mobil Balap: " + kecepatanMaksimal); // 350
        System.out.println("Kecepatan Normal Kendaraan: " + super.kecepatanMaksimal); // 120
    }
}
```

3.2 super untuk Memanggil Method yang Di-Override

Saat subclass melakukan *override* (menimpa) method dari superclass, panggil method asli tersebut menggunakan super.namaMethod().

```
class Karyawan {
    public void tampilkanJobDesk() {
        System.out.println("Tugas: Hadir ke kantor dan bekerja.");
    }
}

class Programmer extends Karyawan {
    @Override
    public void tampilkanJobDesk() {
        super.tampilkanJobDesk(); // Memanggil method Karyawan
        System.out.println("Tugas spesifik: Menulis kode.");
    }
}
```

3.3 super() untuk Memanggil Konstruktor Superclass

Saat objek subclass dibuat, ia otomatis memanggil konstruktor dari superclassnya terlebih dahulu. Jika superclass memiliki konstruktor berparameter, subclass **wajib** memanggilnya menggunakan super(parameter) di baris pertama konstruktornya.

```
class Bentuk {
   String warna;

   public Bentuk(String warna) {
      this.warna = warna;
   }
}

class Lingkaran extends Bentuk {
   double jariJari;

   public Lingkaran(String warna, double jariJari) {
      super(warna); // WAJIB: Memanggil konstruktor superclass di baris PERTAMA
      this.jariJari = jariJari;
   }
```

4. Array & ArrayList

Dalam Java, kita sering kali perlu menyimpan banyak data atau objek sekaligus. Untuk melakukan hal ini, kita bisa menggunakan **Array** konvensional atau menggunakan class **ArrayList** yang disediakan oleh Java.

4.1 Array (Statis)

Array adalah struktur data dasar di Java yang digunakan untuk menyimpan sekumpulan data dengan **tipe yang sama**.

Kelemahan utama dari Array adalah ukurannya yang **statis/tetap (fixed size)**. Sekali kita mendeklarasikan bahwa array tersebut berisi 5 elemen, maka ukurannya tidak bisa ditambah menjadi 6 atau dikurangi.

Sintaks Deklarasi dan Inisialisasi:

```
// Cara 1: Deklarasi ukuran dulu, isi kemudian
   int[] nilaiUjian = new int[3]; // Membuat array tipe int dengan ukuran 3
   nilaiUjian[0] = 85;
   nilaiUjian[1] = 90;
   nilaiUjian[2] = 78;

   // Cara 2: Deklarasi langsung beserta isinya
   String[] namaMahasiswa = {"Andi", "Budi", "Citra"};
```

Contoh Menggunakan Array of Object:

```
class Buku {
    String judul;
    Buku(String judul) { this.judul = judul; }
}

public class MainArray {
    public static void main(String[] args) {
        // Membuat array yang berisi 2 objek Buku
        Buku[] rakBuku = new Buku[2];

        rakBuku[0] = new Buku("Pemrograman Java");
        rakBuku[1] = new Buku("Struktur Data");

        for (int i = 0; i < rakBuku.length; i++) {
            System.out.println(rakBuku[i].judul);
        }
    }
}
```

4.2 ArrayList (Dinamis)

ArrayList adalah bagian dari *Java Collections Framework* (berada di package java.util). Berbeda dengan Array konvensional, ArrayList bersifat **dinamis (resizable)**. Kita bisa menambah dan menghapus data kapan saja, dan ukuran ArrayList akan menyesuaikan secara otomatis.

Catatan Penting ArrayList:

- ArrayList **TIDAK BISA** menyimpan tipe data primitif secara langsung (seperti int, double, boolean). Kita harus menggunakan *Wrapper Class*-nya (seperti Integer, Double, Boolean).
- Harus di-*import* terlebih dahulu: import java.util.ArrayList;

Method-method utama ArrayList:

- .add(data) : Menambahkan data ke baris terakhir.
- .get(index) : Mengambil data pada indeks tertentu.
- .set(index, data): Mengubah isi data pada indeks tertentu.
- .remove(index) : Menghapus data pada indeks tertentu.
- .size() : Menghitung total elemen (mirip .length pada Array).

Contoh Penggunaan ArrayList Dasar:

```
import java.util.ArrayList;

public class MainArrayList {
    public static void main(String[] args) {
        // Membuat ArrayList untuk menyimpan String
        ArrayList<String> daftarMenu = new ArrayList<>();
```

```
        // 1. Menambah data
        daftarMenu.add("Nasi Goreng");
        daftarMenu.add("Mie Ayam");
        daftarMenu.add("Sate Ayam");

        // 2. Mengubah data
        daftarMenu.set(1, "Mie Goreng"); // "Mie Ayam" diganti jadi "Mie Goreng"

        // 3. Menghapus data
        daftarMenu.remove(0); // Menghapus "Nasi Goreng" di index 0

        // 4. Menampilkan data dengan looping
        System.out.println("Total menu saat ini: " + daftarMenu.size());

        for (int i = 0; i < daftarMenu.size(); i++) {
            System.out.println("Menu ke-" + i + ": " + daftarMenu.get(i));
        }
    }
}
```

4.3 Menggabungkan ArrayList dengan Inheritance (Polimorfisme)

Sinergi terbaik dalam OOP adalah ketika kita menggabungkan ArrayList dengan Inheritance. Kita bisa membuat **satu ArrayList bertipe Superclass**, yang di dalamnya dapat diisi oleh berbagai macam objek dari **Subclass**-nya.

Contoh Kasus:

```
import java.util.ArrayList;

    // 1. Superclass
    class Pegawai {
        String nama;
        Pegawai(String nama) { this.nama = nama; }

        public void bekerja() {
            System.out.println(nama + " sedang bekerja.");
        }
    }

    // 2. Subclass A
    class Manager extends Pegawai {
        Manager(String nama) { super(nama); }
```

```
    @Override
    public void bekerja() {
        System.out.println(nama + " (Manager) sedang memimpin rapat.");
    }
}

// 3. Subclass B
class Staff extends Pegawai {
    Staff(String nama) { super(nama); }

    @Override
    public void bekerja() {
        System.out.println(nama + " (Staff) sedang mengerjakan laporan.");
    }
}

// 4. Main Class dengan ArrayList
public class MainPerusahaan {
    public static void main(String[] args) {
        // Mendeklarasikan ArrayList dengan tipe Superclass (Pegawai)
        ArrayList<Pegawai> daftarKaryawan = new ArrayList<>();

        // Kita bisa memasukkan objek Manager maupun Staff ke dalamnya!
        daftarKaryawan.add(new Manager("Pak Budi"));
        daftarKaryawan.add(new Staff("Andi"));
        daftarKaryawan.add(new Staff("Citra"));

        // Looping untuk memanggil method masing-masing
        for (Pegawai p : daftarKaryawan) {
            // Java akan secara cerdas memanggil method override masing-masing subclass
            p.bekerja();
        }
    }
}
```

Output program di atas:

```
Pak Budi (Manager) sedang memimpin rapat.
Andi (Staff) sedang mengerjakan laporan.
Citra (Staff) sedang mengerjakan laporan.
```

LATIHAN PRAKTIKUM (Studi Kasus)

Soal 1: Buatlah sebuah package bernama game agar struktur kode tetap rapi. Di dalamnya, buat sebuah kelas induk (superclass) bernama Hero. Setiap karakter wajib memiliki atribut identitas berupa nama (String), jumlah darah atau health(double), dan kekuatan serangan atau attackPower (double). Buatlah sebuah konstruktor berparameter agar ketiga atribut dasar ini langsung terisi nilainya pada saat objek karakter dibuat.

Soal 2: Pada Class Hero, tambahkan 3 fungsi. Pertama, buat fungsi display() untuk menampilkan status nama, health, dan attackPower karakter. Kedua, buat fungsi berlatih() yang bertugas menambah nilai attackPower karakter sebesar 10 poin, lalu mencetak pesan ke layar bahwa karakter tersebut sedang berlatih. Ketiga, buat fungsi terimaSerangan(double damage) yang bertugas mengurangi nilai health saat ini sejumlah damage yang masuk, sekaligus mencetak mengenai besaran damage yang diterima karakter tersebut.

Soal 3: Rancanglah sebuah subclass baru bernama HeroStrength yang mewarisi Class Hero. Tambahkan satu atribut khusus untuk kelas ini berupa tipe (String) yang diberikan nilai default "Strength". Agar proses penciptaan karakter spesialis ini lancar, buatlah konstruktor HeroStrength untuk mengoper nilai inisialisasi nama, health, dan attackPower langsung.

Soal 4: Override dua fungsi bawaan di dalam kelas HeroStrength. Pertama, override fungsi display() agar karakter ini tidak hanya menampilkan status dasar, tetapi juga mencetak tipe karakternya (yakni "Strength"). Kedua, override mekanisme pertahanan pada fungsi terimaSerangan(double damage). Mengingat pertahanannya yang luar biasa, karakter ini memiliki keistimewaan hanya menerima damage sebesar separuh (50%) dari total damage musuh yang mengenainya.

Soal 5: Di dalam fungsi utama program, Buat dua Hero, hero1 sebagai perwakilan Hero standar dan hero2 sebagai HeroStrength. Tentukan nilai awal nama, health, dan attackPower mereka sesuai kreativitas Anda. Lakukan display awal untuk memastikan data masuk dengan benar. Setelah itu, perintahkan kedua karakter untuk berlatih(). Kemudian, simulasikan mereka berdua tiba-tiba terkena damage sebesar 50). Lalu, panggil kembali fungsi display() untuk kedua karakter.

POST-TEST (Durasi: 40 - 50 Menit)

Sebagai seorang *programmer* di suatu perusahaan , Anda ditugaskan untuk membuat sebuah sistem pendataan penyewaan kendaraan menggunakan konsep OOP. Buatlah sebuah *package* bernama rental. Di dalam *package* tersebut, rancanglah sebuah *superclass* bernama Kendaraan yang menyimpan informasi fundamental berupa nomorPolisi (String), merk (String), tahunProduksi (int), dan hargaSewaDasar (double). Sediakan konstruktor untuk menginisialisasi atribut ini, fungsi displayInfo() untuk menampilkan detail data, serta fungsi hitungHargaSewa(int hari) yang mengembalikan total biaya sewa (hargaSewaDasar dikali hari). **Tambahkan aturan khusus pada fungsi perhitungan tersebut: jika tahunProduksi di bawah tahun 2015, berikan pemotongan (diskon) sebesar 10% dari total sewa dasar.**

Selanjutnya, buatlah dua buah *subclass* bernama Mobil dan Motor yang mewarisi sifat dari Kendaraan. Kelas Mobil memiliki tambahan atribut spesifik kapasitasPenumpang (int). Pada kelas Mobil, Anda harus meng-*override* fungsi hitungHargaSewa(int hari). Aturannya: panggil perhitungan dasar dari *superclass* (gunakan super.hitungHargaSewa), lalu tambahkan biaya asuransi wajib Rp50.000 per hari. **Selain itu, jika kapasitasPenumpang lebih dari 5 orang, tambahkan lagi biaya kebersihan *flat* (sekali bayar/tidak dikali hari) sebesar Rp50.000.** Jangan lupa *override* fungsi displayInfo() untuk menampilkan kapasitas penumpang. Di sisi lain, kelas Motor memiliki tambahan atribut spesifik kapasitasMesin (int, melambangkan CC). Lakukan *override* juga pada fungsi hitungHargaSewa(int hari) untuk motor: panggil perhitungan dasar dari *superclass*, dan **jika kapasitasMesin lebih besar atau sama dengan 250 (CC), kenakan pajak barang mewah tambahan sebesar Rp25.000 per harinya.** *Override* juga fungsi displayInfo() untuk mencetak spesifikasi CC motor tersebut.

Untuk menguji keberhasilan logika sistem Anda, buatlah kelas Main di luar *package* tersebut. Di dalam fungsi utama, deklarasikan sebuah ArrayList dengan tipe Kendaraan. Isikan **4 buah objek** dengan skenario berikut: (1) Mobil lama (di bawah 2015) berkapasitas > 5, (2) Mobil baru (>= 2015) berkapasitas <= 5, (3) Motor ber-CC >= 250, dan (4) Motor standar ber-CC < 250. Terakhir, gunakan perulangan (*looping*) untuk menelusuri setiap objek di dalam koleksi ArrayList, panggil fungsi displayInfo(), lalu cetak "Total Biaya Sewa selama 5 Hari: " beserta hasil pemanggilan fungsi hitungHargaSewa(5). Buktikan bahwa polimorfisme (*ArrayList* + *Inheritance*) dan perhitungan logika kondisional pada tiap subclass berjalan dengan sangat presisi!